

RISC Design

RISC (Reduced Instruction Set Computing) machines grew out of an attempt to do a better job of supporting high-level languages (HLL).

There was believed to be a problem developing called the “semantic gap”, as HLLs became more complex and supported more complex operations

Solving the semantic gap

Two basic approaches were tried to address this problem.

CISC Complex instruction sets with large numbers of addressing modes and special instructions. Implement HLL structures in hardware.

RISC Simplify the instruction set. Do fewer things, but do them fast

The RISC approach

RISC was based on measurement. Benchmark software was investigated to learn the following:

Operations What operations did HLL compilers most often perform?

Operands What kind of operands are most often used?

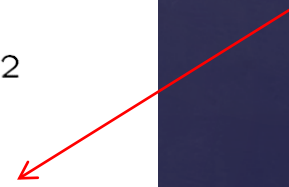
Execution sequencing Sequencing of operations gives a clue to pipeline optimization

Instruction frequency

This table (from the early 1990s) shows the frequencies of various classes of instruction.

Instruction	Pascal	FORTTRAN	Pascal	C
	Scientific	Student	System	System
Assign	74	67	45	38
Loop	4	3	5	3
Call	1	3	15	12
GOTO	2	9	0	3
Other	0	7	6	1

Note the low frequency of everything else



Instruction frequency

Here is the same data, but weighted by instruction execution time.

	Freq		Inst		Mem	
	Pas	C	Pas	C	Pas	C
Assign	45%	38%	13%	13%	14%	14%
Loop	5%	3%	42%	32%	33%	26%
Call	15%	12%	31%	33%	44%	45%
If	29%	43%	11%	21%	7%	13%
Goto	0%	3%	0%	0%	0%	0%
Other	6%	1%	3%	1%	2%	1%

The high per-unit cost of function calls makes them a substantial user of time



Everything else is still extremely rare

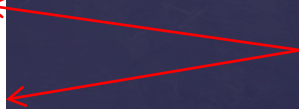


Operand usage

The data for operands looks like this:

	Pascal	C	Avg
Integer const	16%	23%	20%
Scalar	58%	53%	55%
Composite	26%	24%	25%

The bulk of operands are **scalars** or **scalar constants**



Also, more than **80%** of scalars are **local** to the function

Subroutine calls

The high cost of subroutine calls makes them an especially good area for speed-up. Further study yielded what I call the “Rule of Six”:

- 98% of subroutine calls are passed fewer than six arguments
- 92% of subroutines use fewer than six local variables
- Subroutines rarely nest more than six deep

RISC Design

This leads to some architectural ideas for RISC processor. Basically the idea is to put the optimization **where it is needed**. Don't optimize stuff that isn't used. Optimize where the HLL is spending all its time.

- Most “other” instructions are never used. So don't implement them. Use a small instruction set.
- Use **registers** as much as possible
- **Optimize the pipeline** to reduce branch penalties

RISC Loves Registers

Remember the following results:

- A large fraction of HLL programs are **simple assignments**
- A large fraction of data access are **local scalars**
- A large fraction of procedures have a **small number of local scalars and parameters**

Since our **fastest storage is registers**, why not use registers for **all** of this (parameters, locals, etc)?

The Register File

But how can we do that? Registers are hardware. They are **permanent** and **fixed** locations.

Local variables and parameters live on the stack. Their location is **dynamic** and their existence is **ephemeral**.

So how can locals and parameters be registers?

Register Windowing

The solution is called **register windowing**. Consider the following sketch:

[illegible]

Register Windowing

In this example, each subroutine context can see a context that consists of:

Parameters: R0, R1, R2, R3 (R0 would be the return address)

Locals: R3, R4

Temps: R6, R7, R8, R9

Of course the rule of six means you would want to provide **at least six** (and more likely eight) of each type to each context.

Remember our example subroutine call `Func(a,b,c)` from before:

Signature:

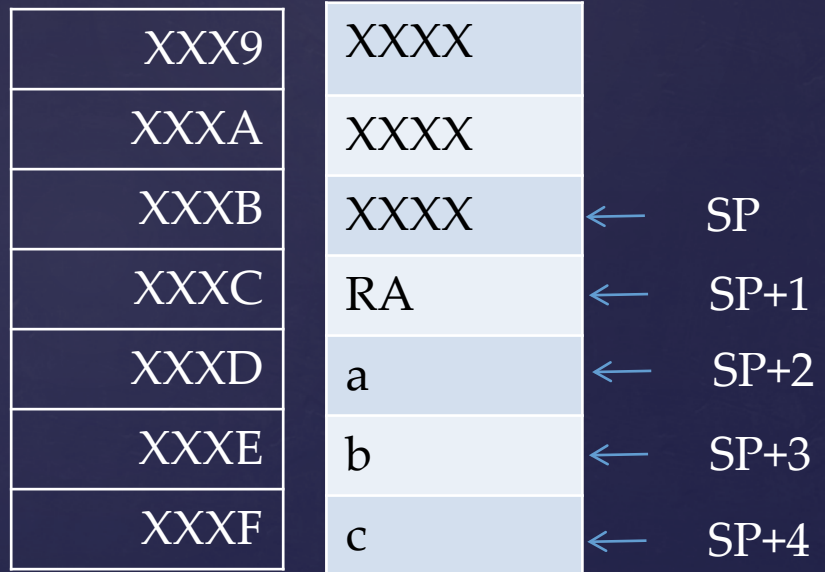
```
void Func(int a, int b, int c)
{
    int x;
    int y;
    int z;

    ...
    x = a+b+c;
}
```

The old way (caller)

We generated the stack frame, like this, assuming i,j are variables in memory

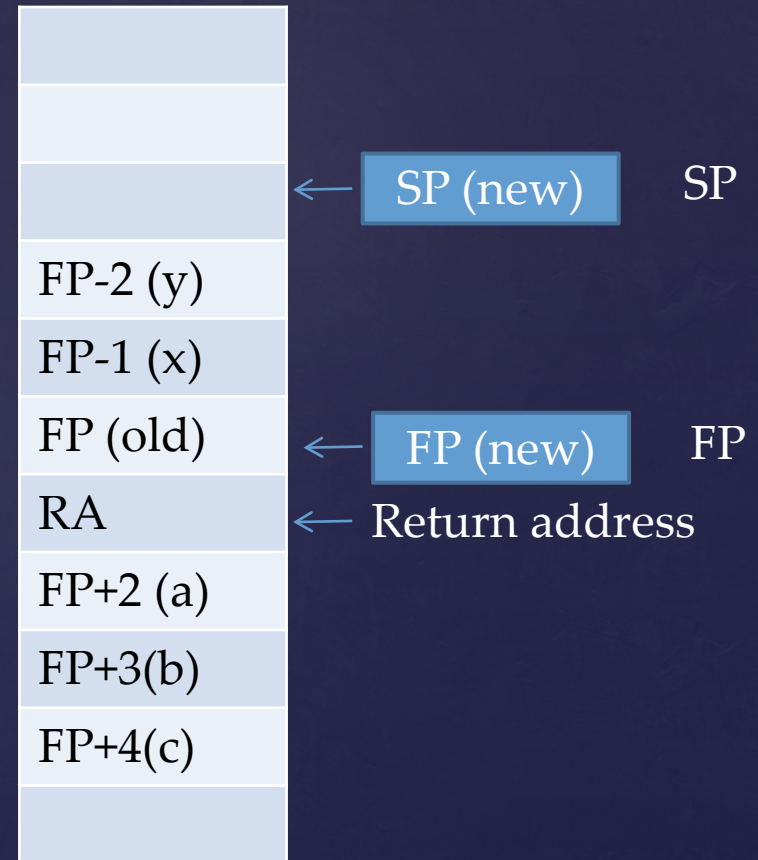
C-code	Assembly
Func(i,j,1);	LOAD R, #1
	PUSH R
	LOAD R, j
	PUSH R
	LOAD R, i
	PUSH R
	JSR Func
	ADD SP, 3



The old way (callee)

Then the caller will do something like this:

C-code	Assembly
int Func (...){	
int x,y;	LINK 2
x=a+b;	LOAD R,FP(2)
	ADD R,FP(3)
	STO R,FP(-1)
...	...
}	UNLINK
	RTS



The RISC way (caller)

In RISC, the local variables i, j, k are actually registers $R4, R5$

C-code	Assembly
Func(i,j,k);	MOV R4,R7
	MOV R5,R8
	LD R9,#1
	JAL R6,Func

Parameters

Return address in register

The RISC way (callee)

What the **callee thinks** it can see is this:

R0 – Return address to caller

R1 – Parameter a

R2 – Parameter b

R3 – Parameter c

R4 – Local x

R5 – Local y

R6 – Temp

R7 – Temp

R8 – Temp

R9 – Temp

C-code	Assembly
int Func (...){	
int x,y;	
x=a+b;	ADD R4,R1,R2
...	...
}	JMP R0

It's not easier for a human to read, but so what? A human's not reading it

Register Files in Practice

In practice, of course, sometimes things don't work:

1. What if the programmer wants **more than six locals**? **Stack**
2. What if the calls are nested **more than six deep**? **Circular window** and **Stack**.
3. What if the programmer wants **global storage**? Set aside some **registers** for globals. Often **R0 is always zero**, for example.

Register File vs Cache

Haven't we just re-invented **cache**? Not exactly

Advantages of **cache**

- Self balancing. Use what you need
- Can store code

Advantages of register window

- Shorter addressing (fewer bits)
- Simpler instruction set (no addressing modes)

RISC vs CISC

We used to fight and call each other names over this issue. Modern computers, as we will see, adopt basically the best of both worlds.

RISC characteristics

Generally, RISC systems have these features:

- One instruction per cycle – No instruction **execution** should never take more than **one ALU operation**
- Register-to-register only – More complex addressing modes are **emulated** in software
- Simple addressing modes – Memory access limited to **LD** and **ST** and perhaps a small set of addressing modes
- Simple instruction formats – **Fixed** positions for opcode, operands, simplify decoding

RISC pipelining

When we first looked at pipelining, we were considering CISC systems that might have two or three arithmetic operations and two or three memory operations per instruction. Pipelines got complicated.

RISC instructions are simple, so RISC pipelines are simple.

We usually model a RISC instruction as consisting of three phases:

I – **Instruction** Fetch

E – **Execute** (compute a result or an address)

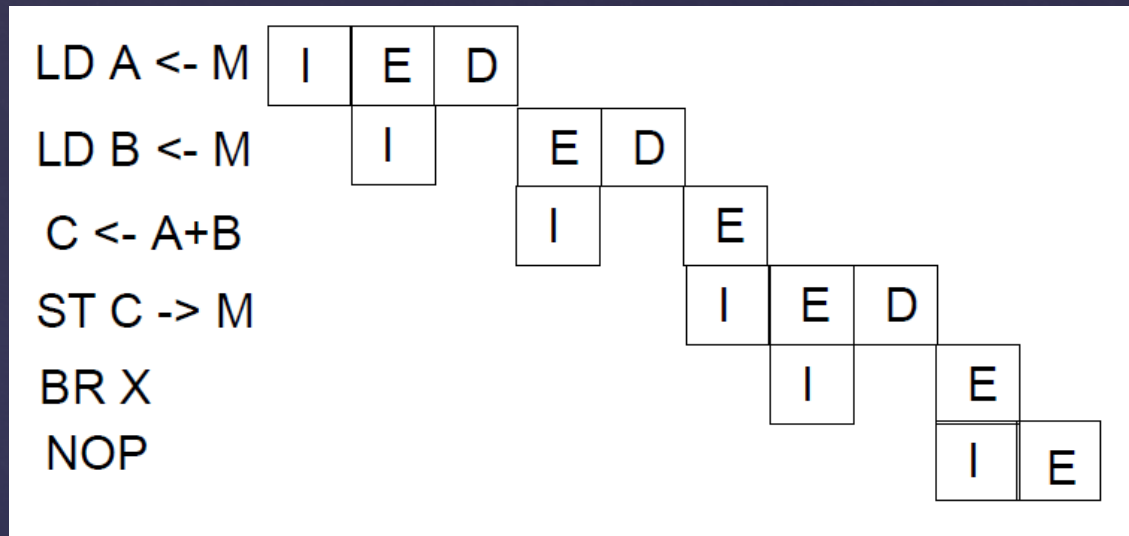
D – Carry out a **data** operation

Pipeline challenges

Remember the challenges we identified with pipelines:

- 1) One memory cycle per instruction: Split I-cache and D-cache, or insert a bubble
- 2) Branch penalties: The pipeline is short. Use delayed branching.
- 3) Dependencies: Are rare. If they happen, insert a bubble

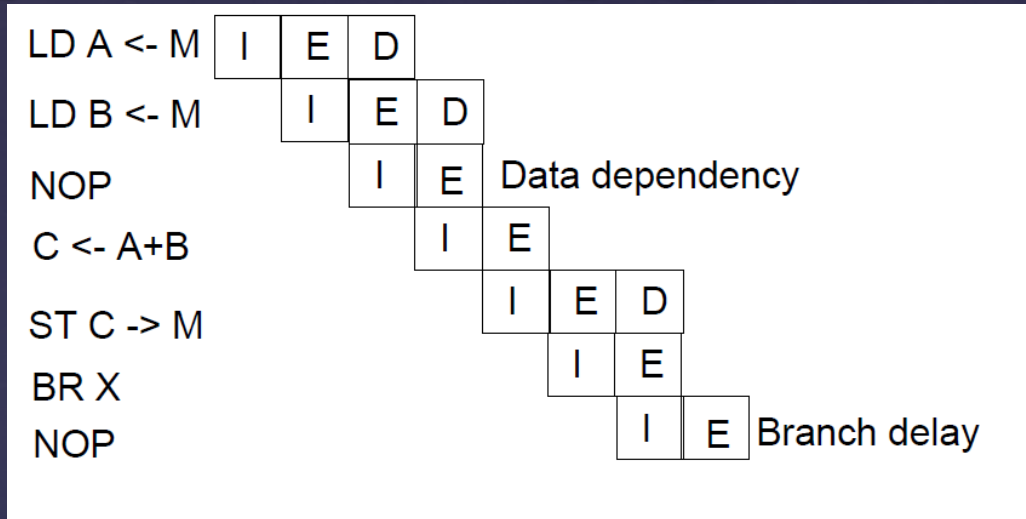
Single cache pipeline



A RISC pipeline with a single cache has **two states**.

- 1) Fetch I(n) and execute I(n-1)
- 2) Load or store.

Split cache pipeline



A RISC pipeline with a split cache has **one state**. Every clock cycle is:
1) Fetch I(n), execute I(n-1), LD or ST for I(n-2)